How Attack Data Is Built Decides Which Attacks Look Detectable: Two Counterexamples from Agent Monitoring

Bipin Rimal
Independent Researcher
bipinrimal314@gmail.com

***Abstract*—Behavioural anomaly detectors for AI agents are often evaluated in two ways: by injecting synthetic attack profiles into agent traces, and by computing AUC-ROC on public trace benchmarks. We show that each choice can decide the answer. First, how the attack data is constructed decides which attacks look detectable. We compare our synthetic perturbation profiles for four OWASP agentic risk categories with the same categories in ATBench, an independently constructed benchmark. For the same three detectors, per-category AUC-ROC differs by up to 0.39. Tool misuse (ASI02) scores 0.51–0.52 under our profiles and 0.82–0.91 on ATBench. Memory and context poisoning (ASI06) errs the other way (0.92–0.98 against 0.80–0.91). The ASI02 gap follows from the generator, which leaves the behavioural structure of each trace unchanged. Second, the TRAIL benchmark has about five negative traces. The Hanley–McNeil 95% interval around an AUC-ROC of 0.577 runs from 0.33 to 0.82, so TRAIL cannot rank detectors, and our earlier observation that transferred detectors beat in-domain ones is inside that noise. Third, a second synthetic prediction, that distributed distillation stays near chance, agrees with a later public disclosure. Nothing distinguished it beforehand from the ASI02 prediction that failed. We recommend treating synthetic profiles as hypotheses, checking them against an independent construction, and reporting an interval for every AUC.**

I. Introduction

Monitoring AI agents with behavioural anomaly detection is attractive. It needs no labels at training time, and it reuses decades of work on insider-threat detection. Evaluating such a monitor is harder than building it. Labelled agent attacks are rare, so two shortcuts are common. The first shortcut injects synthetic attack profiles into benign traces and measures how well the detector finds them, one risk category at a time. The second shortcut reports AUC-ROC on a public trace benchmark.

This paper reports what happened when we checked both shortcuts in our own earlier work [1]. That work used synthetic profiles to name tool misuse (OWASP ASI02 [2]) as a structural blind spot. It also reported that a detector transferred from another domain scored higher on TRAIL [3] than one trained on TRAIL itself. Neither claim survives.

We make three points.

1) **Construction decides detectability.** Two independent constructions of the same OWASP categories disagree by up to 0.39 AUC-ROC per category, in both directions (Section III).
2) **TRAIL cannot rank detectors.** With about five negatives, the 95% interval on a mid-range AUC-ROC spans roughly ± 0.24 (Section IV).
3) **The failure is invisible from inside.** A synthetic prediction that was later corroborated looked the same, beforehand, as the one that failed (Section V).

These are counterexamples, not a general law. Each one is small and specific. We think their value is that each is easy to check and easy to repeat in other evaluations.

II. Setup

Features. Each trace becomes a 20-dimensional Unified Behavioural Feature Schema (UBFS) vector in seven groups: temporal, frequency, volume, scope, sequence, deviation and privilege [1]. The features describe behaviour only: timing, counts, tool breadth, action entropy, transition novelty and similar. No feature reads the text of a prompt, a tool argument or an output.

Detectors. We use Isolation Forest (IF) [4] with 200 trees, an LSTM autoencoder (LSTM) [5], and a deep-clustering autoencoder (DC) [6]. Each is trained on benign traces only and scores all test traces. Every result is a mean over five seeds.

Synthetic construction. We inject attack profiles into TRAIL traces. Each profile is a rule that edits a benign trace to match our idea of one OWASP category. For example, the goal-hijack profile (ASI01) inserts unfamiliar tool calls mid-trace, and the memory-poisoning profile (ASI06) duplicates the session's spans several times.

Independent construction. ATBench [7] provides 500 multi-turn tool-calling trajectories with safety labels and fine-grained risk categories, which we map to OWASP ASI codes. ATBench is not a record of real incidents. Its authors construct the trajectories and then filter them with rules, language models and a human audit. We use it as a second construction of the same categories, built by people who did not see our profiles.

Trace benchmark. TRAIL [3] contains agent traces from GAIA and SWE-bench tasks with annotated errors. About 97% of traces contain an error, so about five traces are negatives. TRAIL was designed for error localisation, not for trace-level detection, so our use of it is outside its design intent.

TABLE I
PER-CATEGORY AUC-ROC UNDER TWO CONSTRUCTIONS OF THE SAME OWASP CATEGORIES (UBFS-20, MEAN OF FIVE SEEDS). SYN.: OUR PERTURBATION PROFILES ON TRAIL. ATB.: ATBENCH. Δ = ATB. − SYN.

	IF			LSTM			DC		
Cat.	Syn.	ATB.	Δ	Syn.	ATB.	Δ	Syn.	ATB.	Δ
ASI01	.642	.843	+.20	.637	.785	+.15	.621	.894	+.27
ASI02	.520	.871	**+.35**	.510	.823	**+.31**	.524	.910	**+.39**
ASI06	.920	.869	−.05	.975	.798	**−.18**	.981	.906	−.08
ASI10	.729	.764	+.04	.696	.714	+.02	.730	.850	+.12

III. CONSTRUCTION DECIDES DETECTABILITY

Table I compares the four categories that both constructions share. The detectors and features are identical. Only the attack data changes.

The two constructions disagree in both directions. Under our profiles, tool misuse (ASI02) sits near chance for all three detectors (0.51–0.52). On ATBench the same detectors score 0.82–0.91 on the same category. Memory and context poisoning (ASI06) errs the other way: our profiles make it look easier (0.92–0.98) than ATBench does (0.80–0.91). The largest single gap is 0.39 (DC, ASI02). The gaps are large compared with seed variation: the synthetic ASI02 standard deviation across seeds is at most 0.037.

The ASI02 gap follows from the generator. Our ASI02 profile does not add, remove or reorder any span, and it does not change any tool name. It only rescales existing numeric span attributes, such as token counts, by a small random factor. The count, scope and sequence features therefore do not change, and only volume-type features move, slightly. A behaviour-only detector has almost nothing to see, whatever real tool misuse looks like. The near-chance score measures our definition of the category, not the category.

A second test agrees. Adding eight semantic features to the schema (UBFS-28) leaves synthetic ASI02 near chance: DC moves from 0.537 to 0.559, and IF moves from 0.514 to 0.494. A profile that changes nothing a feature can read stays invisible when more features are added. On ATBench, the same semantic features raise ASI02 further (IF: 0.954; Appendix A).

What the comparison does not show. ATBench is also a construction. The disagreement shows that the answer depends on who builds the attack data. It does not show which construction is closer to attacks in deployment. Our earlier write-up reported a perfect rank inversion over three shared categories. With the category mapping corrected, four categories are shared, and the Spearman correlation between the two rankings is -0.40. With $n = 4$ no rank correlation can reach significance (the smallest attainable exact two-sided p is 0.083), so we do not use it as evidence. The evidence is the size of the per-category gaps and the known mechanism.

IV. TRAIL CANNOT RANK DETECTORS

AUC-ROC is the probability that a random positive scores above a random negative [8]. With about five negatives, every AUC on TRAIL is an average over the ranks of about five traces. The Hanley–McNeil standard error [8] gives a 95% interval of 0.33–0.82 around IF's in-domain score of 0.577, about ± 0.24.

TABLE II
AUC-ROC ON TRAIL BY TRAINING DATA (MEAN $\pm$ STD OVER FIVE SEEDS). THE HANLEY–MCNEIL 95% INTERVAL AROUND 0.577 IS 0.33–0.82.

Trained on	IF	DC
TRAIL (in-domain)	$0.577 \pm .021$	$0.897 \pm .021$
TRACE (transfer)	$0.614 \pm .007$	$0.631 \pm .030$
ATBench (transfer)	$0.774 \pm .009$	$0.734 \pm .050$

Table II shows the pattern we earlier read as a finding. An IF trained on TRACE [9] (0.614) or on ATBench (0.774) scores higher on TRAIL than an IF trained on TRAIL (0.577). Both transferred scores lie inside the interval around the in-domain score. The pattern is therefore not evidence of anything. It also does not hold for DC, whose in-domain score (0.897) is the highest in the table.

Seed variation hides the problem. The seed standard deviations in Table II are small (0.007–0.050). They measure the randomness of model training. They do not measure the sampling of the five negatives, which is the dominant source of error. A table of means and seed deviations therefore looks precise when it is not.

The cause is few negatives, not prevalence as such. AUC-ROC does not depend on class balance in principle [8], [10]. The problem is that about five negative traces make the statistic too noisy to compare anything. With so few negatives, any nuisance property they share, such as trace length or source task, can move the score. AUC-PR does not help here: IF's in-domain AUC-PR is 0.982, close to what any scorer achieves when about 97% of traces are positive [11].

V. THE FAILURE IS INVISIBLE FROM INSIDE

Our earlier work made a second synthetic prediction of the same kind. It modelled distributed model distillation (HYDRA): extraction traffic spread over many accounts, so that each account looks normal. All three detectors stay near chance on this profile (IF 0.538, LSTM 0.538, DC 0.540). By contrast, a focused single-account extraction profile is easy to detect (IF 0.995, LSTM 0.999, DC 0.936).

A later public disclosure [12] described distillation campaigns spread across many accounts, and detection that relied on signals across accounts rather than on per-account behaviour. This agrees with the HYDRA prediction. It is corroboration in kind, not a measurement: no labelled benchmark of distributed extraction was available to us.

The ASI02 prediction and the HYDRA prediction came from the same pipeline. Both were near-chance scores on profiles we defined. Both had a plausible story attached. One agreed with later evidence and one did not. Nothing inside the synthetic experiment separated them. We had an argument for HYDRA that does not rest on the profile (a per-account

detector sees each account alone, and coordination exists only across accounts), but we also had a plausible argument for ASI02 before ATBench contradicted it. A synthetic result alone could not tell us which argument to trust.

VI. Related Work

Synthetic data in detection. Critiques of the DARPA intrusion-detection evaluations showed that simulated traffic gives optimistic and artefact-driven results [13], [14], and these pitfalls remain common in security ML [15]. Those studies report inflated absolute scores. We report a per-category disagreement that runs in both directions. In AI control, Li et al. find that monitors trained on prompt-elicited reward hacking fail on hacking that arises during training [16]. Loughridge et al. report the opposite case, where calibrated synthetic attacks did transfer [17]. Our ASI02 profile was not calibrated against anything, which may explain the difference.

Evaluation validity. Zavrak shows that random splits inflate AUROC by up to 26 points on agent tool-call detection, that metadata-only features plateau near 0.64 AUROC, and that content-aware features reach about 0.89 or higher [18]. A validity audit of agent-safety benchmarks shows that an always-positive baseline can beat several models [19]. Wu and Keogh show how flawed anomaly benchmarks create an illusion of progress [20]. We found no prior report of a per-category disagreement between injected profiles and an independent benchmark, or of TRAIL's few negatives as a limit on detector ranking. Our search was not exhaustive.

Prior work by the author. This paper revises results from the author's earlier paper [1], which reported the synthetic ASI02 blind spot and the TRAIL transfer result. Both are withdrawn here.

VII. Limitations

Behaviour-only features. UBFS reads no content. Content-aware detectors reach about 0.89 AUROC or higher on related data [18], so our detectors are weak by current standards. The construction effect may be smaller or larger for stronger detectors.

No LLM-monitor baseline and no trivial baselines. We did not compare against a language-model monitor. We did not report always-positive, trace-length or source-task baselines, so we cannot say how much of any TRAIL score a trivial feature explains.

Random splits. Our splits are random, not task-disjoint. Zavrak shows that this choice alone can inflate AUROC [18].

Single constructions. Each side of Table I is one construction: one profile set and one benchmark. Only four categories are shared, and ASI02 has only 29 positive trajectories in ATBench. Other generators or benchmarks may agree with each other.

Intervals. We give a Hanley–McNeil interval for one TRAIL score only. Every AUC in this paper should carry an interval.

Lost insider-threat pipeline. Our earlier work also transferred detectors from the CMU-CERT insider-threat data. The feature-loading code for that data was never committed and is lost, so those numbers cannot be rerun. We do not use them here.

Reproduction. A fresh rerun of the original thirteen experiments reproduced the published tables for 10 of them, including every non-CERT number in the baseline and transfer experiments.

VIII. Conclusion

How the attack data is built decided which attacks looked detectable in our evaluation, and a widely used trace benchmark had too few negatives to rank our detectors. Neither problem was visible from inside the experiment. Three practices follow. Treat a synthetic per-category score as a hypothesis about the profile. Check it against a construction built by someone else before calling a category a blind spot. Report an interval for every AUC, and count the negatives first.

References

[1] B. Rimal, "From rogue employees to rogue agents: Repurposing insider threat detection for AI agent governance," SSRN Working Paper 6355658, https://papers.ssrn.com/sol3/papers.cfm?abstract_id=6355658, 2026.

[2] OWASP Foundation, "OWASP top 10 for agentic applications," https://owasp.org/www-project-top-10-for-large-language-model-applications/llm-top-10-governance-doc/GOV300_Agentic_AI_Threats, 2024, accessed: 2025-12-15.

[3] Patronus AI, "TRAIL: Trace and inspect agent logs," in *Patronus AI Technical Report*, 2024.

[4] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *2008 Eighth IEEE International Conference on Data Mining*. IEEE, 2008, pp. 413–422.

[5] P. Malhotra, A. Ramakrishnan, G. Anand, L. Vig, P. Agarwal, and G. Shroff, "Lstm-based encoder-decoder for multi-sensor anomaly detection," in *ICML 2016 Anomaly Detection Workshop*, 2016.

[6] S. Xie, H. Luo, and L. Li, "A comprehensive survey on deep clustering: Taxonomy, challenges, and future directions," *ACM Computing Surveys*, vol. 56, no. 4, pp. 1–38, 2024.

[7] AI45Research, "ATBench: A multi-turn tool-centric safety benchmark for ai agents," *arXiv preprint arXiv:2604.02022*, 2026, available at https://huggingface.co/datasets/AI45Research/ATBench.

[8] J. A. Hanley and B. J. McNeil, "The meaning and use of the area under a receiver operating characteristic (ROC) curve," *Radiology*, vol. 143, no. 1, pp. 29–36, 1982.

[9] J. Pan, T. He, and S. Thwaites, "Spontaneous reward hacking in iterative self-refinement," in *Patronus AI Technical Report*, 2024, arXiv:2505.08638.

[10] McDermott *et al.*, "A closer look at AUROC and AUPRC under class imbalance," in *Advances in Neural Information Processing Systems*, 2024, arXiv:2401.06091.

[11] T. Saito and M. Rehmsmeier, "The precision-recall plot is more informative than the ROC plot when evaluating binary classifiers on imbalanced datasets," *PLoS ONE*, vol. 10, no. 3, p. e0118432, 2015.

[12] Anthropic, "Detecting and countering malicious uses of Claude: February 2026," https://www.anthropic.com/research/detecting-malicious-uses-feb-2026, 2026, disclosed model distillation attacks by DeepSeek, Moonshot AI, MiniMax (24,000+ accounts, 16M+ exchanges) and AI-orchestrated espionage by GTG-1002.

[13] J. McHugh, "Testing intrusion detection systems: A critique of the 1998 and 1999 DARPA intrusion detection system evaluations as performed by Lincoln Laboratory," *ACM Transactions on Information and System Security*, vol. 3, no. 4, pp. 262–294, 2000.

[14] M. V. Mahoney and P. K. Chan, "An analysis of the 1999 DARPA/Lincoln Laboratory evaluation data for network anomaly detection," in *Recent Advances in Intrusion Detection (RAID)*, ser. LNCS, vol. 2820, 2003, pp. 220–237.

[15] D. Arp *et al.*, "Dos and don'ts of machine learning in computer security," in *USENIX Security Symposium*, 2022, arXiv:2010.09470.

[16] Li *et al.*, "Do prompt-elicited trajectories reflect training-time reward hacking?" *arXiv preprint arXiv:2604.23488*, 2026.
[17] Loughridge *et al.*, "Optimizing AI agent attacks with synthetic data," *arXiv preprint arXiv:2511.02823*, 2025.
[18] Zavrak, "Content-aware attack detection in LLM agent tool-call traffic," *arXiv preprint arXiv:2605.11053*, 2026.
[19] "Safety, or just capability? A validity audit of agent-safety benchmarks," *arXiv preprint arXiv:2607.28685*, 2026.
[20] R. Wu and E. Keogh, "Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress," *arXiv preprint arXiv:2009.13807*, 2020.

APPENDIX

A. Full ATBench Results

Table III gives per-category AUC-ROC on ATBench for all six mapped categories, with the 20-dimensional schema and with eight added semantic features (UBFS-28). ASI04 and ASI08 have no synthetic profile.

TABLE III
ATBENCH PER-CATEGORY AUC-ROC (MEAN OF FIVE SEEDS).

Model	Feat.	ASI01	ASI02	ASI04	ASI06	ASI08	ASI10
IF	20	.843	.871	.848	.869	.776	.764
	28	.943	.954	.927	.939	.909	.877
LSTM	20	.785	.823	.801	.798	.733	.714
	28	.857	.876	.845	.825	.819	.823
DC	20	.894	.910	.897	.906	.858	.850
	28	.926	.933	.909	.910	.895	.873

B. Interval Method

For an AUC A estimated from n_p positives and n_n negatives, Hanley and McNeil [8] give

$$\mathrm{SE}^2 = \big[A(1-A) + (n_p-1)(Q_1 - A^2) + (n_n-1)(Q_2 - A^2)\big]/(n_p\, n_n),$$

with $Q_1 = A/(2-A)$ and $Q_2 = 2A^2/(1+A)$. The 95% interval is $A \pm 1.96\,\mathrm{SE}$. With n_n near five, each negative takes part in a large share of all positive–negative pairs, so the interval stays wide however many positives there are.

C. Synthetic Profiles

Each profile edits a benign TRAIL trace. ASI01 (goal hijack) inserts unfamiliar tool calls mid-trace. ASI02 (tool misuse) rescales numeric span attributes and makes no structural change. ASI06 (memory and context poisoning) duplicates the session's spans. ASI10 (rogue agent) adds divergent actions and privilege signals. The code is in `src/data/synthetic_generator.py`.